

Documentação Técnica

Sistema de Controle de Acesso Biométrico — 5º CTA

5º CTA

Versão 1.0 — Junho de 2026

Sumário

1. Visão Geral	2
1.1 Propósito do sistema	2
1.2 Modelo de implantação	3
1.3 Decisões de design centrais	3
2. Stack Tecnológica	4
2.1 Plataforma e linguagem	4
2.2 Bibliotecas de back-end (Int1)	4
2.3 Bibliotecas do Worker (Int2)	6
2.4 Bibliotecas da infraestrutura (Int4)	6
2.5 Front-end (Int3)	7
3. Arquitetura	7
3.1 Os quatro projetos	7
3.2 Como os projetos se comunicam	8
3.3 O modelo de banco compartilhado	9
4. Banco de Dados	9
4.1 Entidades de negócio	10
4.2 Tabelas de junção e fila	11
4.3 Tabelas de pools e configuração	11
4.4 Migrações e seeding	12
5. Backend — Int1 (API)	12
5.1 Camadas e responsabilidades	12
5.2 Autenticação JWT	13
5.3 Controllers e endpoints	13
5.4 Repositórios	13
5.5 Jobs agendados (Hangfire)	14
5.6 Criptografia (AES)	15
6. Frontend — Int3 (Painel Blazor)	15
6.1 Por que Blazor Server	15
6.2 O ITokenStore e o ciclo de vida do circuito	16
6.3 Páginas e roteamento	16
6.4 JavaScript interop	17
7. Worker — Int2 (Serviço de Background)	17
7.1 BackgroundService principal	17
7.2 Camada de conexão (AnvizConnector)	18
7.3 Camada de processamento (EventProcessor)	18
7.4 Sincronizadores periódicos	19
8. Infraestrutura — Int4	19
8.1 CameraService e integração ONVIF	19
8.2 ExportService (CSV e PDF)	20
9. Fluxos de Negócio	20
9.1 Cadastro de pessoa	20

9.2 Vínculo de pessoa a ambiente	21
9.3 Primeiro acesso e enroll biométrico	21
9.4 Acesso normal por digital	22
9.5 Acesso normal por senha	22
9.6 Acesso negado	22
9.7 Alteração de modo de acesso	23
9.8 Inativação automática e limpeza periódica	23
10. Hardware T50M Anviz	23
10.1 Especificações	23
10.2 Protocolo OAE	24
10.3 Operação local no dispositivo	24
11. Segurança	25
11.1 Estratégia em camadas	25
11.2 Armazenamento de senhas	25
11.3 Autorização nas APIs	26
11.4 Gestão de administradores	26
12. Integrações Externas	27
12.1 Email via servidor SMTP	27
12.2 Câmeras IP via ONVIF e RTSP	27
13. Testes Automatizados	28
14. Build, Empacotamento e Execução	29
Pré-requisitos	29
Build	29
Execução em desenvolvimento	29
Execução em produção	29
Apêndice A — Glossário	29
Apêndice B — Códigos de motivo de negação	30

1. Visão Geral

1.1 Propósito do sistema

O Sistema de Controle de Acesso Biométrico do 5º CTA foi desenvolvido para gerenciar a autenticação e o registro de entrada de pessoas autorizadas em ambientes físicos controlados da unidade militar. O sistema substitui livros de presença manuais e fichas de cadastro avulsas por uma plataforma única e auditável, onde cada acesso liberado ou negado fica gravado com data, hora, ambiente, método de autenticação e — quando disponível — o trecho de vídeo correspondente das câmeras IP do local.

A motivação central é dupla. Por um lado, há a necessidade operacional de saber quem entrou onde e quando, com precisão de segundos, para fins de investigação retrospectiva (por exemplo, identificar quem teve acesso à armaria em determinada noite). Por outro, há a exigência de segurança: o controle por biometria mais senha numérica de seis dígitos elimina cenários em que um cartão de acesso roubado libera ambientes restritos, e a gravação automática associada a cada tentativa garante material probatório.

O sistema atende administradores cadastrados — tipicamente militares com cargo de oficial ou subtenente responsável por cadastrar, gerenciar e auditar o pessoal autorizado. Os usuários finais (pessoas que efetivamente passam pelas portas controladas) não interagem com o painel; sua interface é o próprio dispositivo T50M Anviz instalado em cada porta.

1.2 Modelo de implantação

A topologia de implantação é deliberadamente simples: um único servidor (Windows Server ou Ubuntu, instalado dentro do CTA) hospeda os três processos da aplicação. O banco de dados é um arquivo SQLite local nesse servidor, sem necessidade de SGBD separado. Os dispositivos T50M e câmeras IP residem na mesma rede local que o servidor, conectados por cabo Ethernet a um switch interno.

Essa escolha de implantação local — sem dependência de Internet, sem cloud, sem servidor remoto — é proposital. O ambiente militar exige que dados sensíveis (CPFs, biometria, histórico de acessos) jamais saiam da rede da unidade. O único tráfego que cruza o firewall corporativo do CTA é a saída para o servidor SMTP institucional, e mesmo isso é opcional (sem SMTP, o sistema simplesmente não envia emails, e as credenciais são entregues manualmente).

A consequência arquitetural disso é que o sistema precisa funcionar em uma máquina relativamente modesta — dois núcleos, quatro gigabytes de RAM e SQLite são suficientes para a carga prevista de algumas centenas de pessoas e alguns milhares de acessos por mês. Caso a unidade cresça, a migração para SQL Server é direta (o Entity Framework Core abstrai o SGBD).

1.3 Decisões de design centrais

Algumas decisões moldam todo o resto do sistema e merecem destaque desde o início, porque explicam por que certas escolhas aparentemente óbvias não foram feitas.

A primeira é a **divisão em três processos separados** em vez de um monolito. O backend (Int1), o painel web (Int3) e o serviço de hardware (Int2) rodam como executáveis independentes. Isso permite reiniciar o painel sem derrubar o processamento dos T50Ms, e vice-versa. Para o cliente final, instalar é trivial — basta subir os três como serviços do Windows ou unidades do systemd — mas internamente cada um tem responsabilidade clara.

A segunda é o **compartilhamento de banco via referência de projeto**, e não via API HTTP. O painel Blazor (Int3) e o Worker (Int2) acessam o banco SQLite diretamente, usando os mesmos repositórios definidos no Int1, importados como `ProjectReference`. Isso elimina serialização desnecessária, garante consistência transacional e simplifica o código. A API HTTP do Int1 existe apenas para o que precisa atravessar o limite do processo — autenticação JWT, reenvio de email, exportação de PDF — porque esses casos envolvem o painel chamando a API por motivos práticos (autenticação centralizada, geração assíncrona de relatórios).

A terceira é o **gerenciamento de administradores via banco direto, não pela tela**. Não há, propositalmente, formulário para criar admin no painel. Quem precisa criar um administrador novo abre o banco SQLite com o DBeaver e executa um `INSERT` manual após gerar o hash BCrypt da senha. A justificativa é a sensibilidade do contexto militar: se a UI permitisse criar admin, um administrador comprometido conseguiria criar outros administradores para si sem deixar rastro claro. Forçando o caminho via banco, garante-se que quem cria admin já tem acesso físico ao servidor — uma camada extra de proteção contra escalada de privilégio.

A quarta é o **uso de senhas pré-geradas em pool**. Em vez de o sistema sortear uma senha numérica no momento do cadastro, há uma tabela `SenhaDisponivel` com novecentas mil senhas pré-criadas (de 100000 a 999999, excluindo triviais como 123456). O cadastro de pessoa marca uma senha como em uso e a vincula à pessoa. Isso evita colisões impossíveis (duas pessoas com a mesma senha), garante distribuição uniforme e permite verificar de

antemão quais senhas estão livres. O mesmo padrão é aplicado ao código de usuário do T50M.

2. Stack Tecnológica

2.1 Plataforma e linguagem

Todo o sistema é escrito em **C# 12** rodando em **.NET 8**, a versão de suporte de longo prazo (LTS) da Microsoft com janela de suporte até novembro de 2026. Essa unificação tem dois benefícios práticos: a equipe escreve uma única linguagem ponta a ponta (backend, frontend Blazor, worker), e o deploy fica trivial — basta o runtime do .NET 8 instalado no servidor para executar tudo.

O **SQLite** versão 3.x foi escolhido como banco de dados pela natureza do ambiente. É um banco serverless: o arquivo banco.db é o banco inteiro, sem processo separado, sem porta de rede, sem configuração de SGBD. Backup é cópia de arquivo. Restauração é trocar o arquivo. Para o volume previsto — centenas de pessoas cadastradas, alguns milhares de acessos por mês, dezenas de milhares de tentativas armazenadas — o SQLite supera com folga, especialmente porque o sistema usa o modo WAL (Write-Ahead Logging), que permite leituras concorrentes durante escritas.

A combinação .NET 8 + SQLite é OS-agnóstica, o que significa que o servidor da unidade pode rodar tanto Windows Server quanto Ubuntu sem alteração de código. Na prática, qualquer versão recente do Windows ou distribuição Linux com o runtime .NET 8 instalado é suficiente.

2.2 Bibliotecas de back-end (Int1)

O projeto Int1, alojado em Banco/WebAbil8-Sistema_Verificação_dupla.slnx/, é a alma do sistema e concentra a maior quantidade de bibliotecas externas. Cada uma resolve um problema específico que seria inviável (ou pelo menos custoso) reescrever do zero.

Entity Framework Core 8.0.7, junto com o provider **Microsoft.EntityFrameworkCore.Sqlite** e os pacotes **.Design** e **.Tools**, é o ORM (Object-Relational Mapping) que traduz objetos C# em comandos SQL e vice-versa. Em vez de escrever queries SQL manualmente, definimos classes C# (entidades como Pessoa, Ambiente, Camera) e o EF Core cuida das migrações, das queries via LINQ e do controle de transações. Usamos métodos como `db.Pessoas.Where(p => p.Status == "ativo").Include(p => p.Ambientes).ToList()` para fazer joins (eager loading), `db.SaveChangesAsync()` para persistir mudanças e `db.Database.EnsureCreated()` para criar o banco no startup quando ele ainda não existe. A escolha de EF Core sobre alternativas mais leves como Dapper se justifica pela produtividade — em um sistema com quinze entidades e operações CRUD intensivas, escrever SQL manual seria contraproducente.

BCrypt.Net-Next 4.1.0 resolve o problema de armazenamento seguro de senhas. Nunca guardamos senhas em texto plano no banco. A função `BCrypt.Net.BCrypt.HashPassword(senha, workFactor: 10)` gera um hash irreversível com sal automático e fator de custo configurável — fator 10 significa que cada operação de hash leva por volta de cem milissegundos, o que torna ataques de força bruta computacionalmente inviáveis. No login, `BCrypt.Net.BCrypt.Verify(senhaInformada, hashDoBanco)` confere se a senha digitada bate com o hash armazenado, sem nunca recuperar o texto original. Esse padrão se aplica tanto às senhas dos administradores (usadas no login do painel) quanto às senhas das pessoas cadastradas (usadas no login no T50M).

Hangfire (pacotes `Hangfire.Core`, `Hangfire.AspNetCore` e `Hangfire.MemoryStorage` versão 1.8.14) é o framework de jobs em background. Precisamos executar duas tarefas recorrentes todos os dias às 03:00 UTC sem interferir no funcionamento normal do servidor: limpar tentativas de acesso e logs cuja data de retenção expirou, e inativar pessoas que não acessaram nada nos últimos vinte e quatro meses. O Hangfire faz isso de forma robusta, com retry automático em caso de falha, persistência das tarefas (sobrevivem a restart) e dashboard web em `/hangfire` para acompanhar status. Registramos os jobs no startup com `RecurringJob.AddOrUpdate<InativarUsuariosInativos2AnosJob>("inativar-usuarios", j => j.Executar(), "0 3 * * *")`. O pacote `MemoryStorage` é uma escolha consciente para evitar uma tabela adicional no SQLite — perder o histórico de execuções de jobs em caso de reboot é aceitável, já que os jobs são idempotentes (se não rodaram ontem, rodam hoje sem dano).

MailKit 4.7.1.1 é o cliente SMTP usado para enviar credenciais por email. O cliente clássico do .NET (`System.Net.Mail.SmtpClient`) está oficialmente obsoleto desde anos atrás, e MailKit é a substituta de fato — suporta STARTTLS, SSL puro, autenticação moderna e roda em qualquer servidor SMTP padrão (Zimbra do EBmail, Office 365, Gmail). Quando uma pessoa é cadastrada, o `PersonController.ReenviarCredenciais` chama uma sequência de métodos: `new MimeMessage()` para construir o email, `msg.From.Add(new MailboxAddress(...))` e `msg.To.Add(...)` para definir os endpoints, `msg.Body = new TextPart("html") { Text = corpoHtml }` para o corpo HTML, e em seguida `using var smtp = new SmtpClient(); smtp.ConnectAsync(host, port, SecureSocketOptions.StartTls); smtp.AuthenticateAsync(user, pass); smtp.SendAsync(msg); smtp.DisconnectAsync(true)`. Toda a configuração (host, porta, usuário, senha) é lida de variáveis de ambiente do sistema operacional, nunca do `appsettings.json`.

Microsoft.AspNetCore.Authentication.JwtBearer 8.0.0 combinado com **System.IdentityModel.Tokens.Jwt 7.6.0** implementa a autenticação por tokens JWT (JSON Web Token). Quando um administrador faz login no painel, o `AuthController.Login` valida a senha (com `BCrypt`), e então gera um token assinado contendo o `adminId` e o `nomeCompleto` como claims, usando `new JwtSecurityToken(issuer, audience, claims, expires: DateTime.UtcNow.AddHours(8), signingCredentials)` e em seguida `new JwtSecurityTokenHandler().WriteToken(token)`. O token vai no header `Authorization: Bearer <token>` de toda requisição subsequente. O atributo `[Authorize]` aplicado a todos os controllers (exceto `AuthController` e `HealthController`) garante que requisições sem token válido são rejeitadas com 401. JWT foi escolhido sobre cookies de sessão pela natureza stateless — não precisamos manter estado de sessão no servidor, e o token expira sozinho em oito horas.

Serilog (pacotes `Serilog.AspNetCore` 8.0.3, `Serilog.Settings.Configuration` 8.0.4 e `Serilog.Sinks.Console` 6.1.1) é o framework de logging estruturado. Logs gerados com `_logger.LogInformation("Pessoa {Id} cadastrada com sucesso", pessoaId)` saem com timestamps, níveis e propriedades nomeadas — o que facilita imensamente o diagnóstico quando o cliente reporta um problema. A configuração fica no `appsettings.json` na seção `Serilog`, definindo nível mínimo (Information em produção, Debug em desenvolvimento), sinks (saída para console e arquivo) e enrichers.

QuestPDF 2024.3.4 gera arquivos PDF programaticamente. Em vez de templates HTML convertidos para PDF (abordagem propensa a falhas com layouts complexos), QuestPDF usa uma API fluente em C# para descrever o documento. O `ExportService` no `Int4` (chamado pelo `ExportController` do `Int1`) produz quatro tipos de relatório: histórico de acessos, logs de administrador, lista de pessoas e relatório completo de ambiente. A escolha por PDF programático em vez de Word, LibreOffice ou similar é prática — não exige nenhuma dependência adicional no servidor.

Swashbuckle.AspNetCore 10.1.7 gera a documentação OpenAPI (Swagger UI) automá-

tica para a API. Acessando <http://localhost:5018/swagger>, qualquer desenvolvedor vê todos os endpoints listados, com schemas, parâmetros, autenticação e botão “Try it out” para testar diretamente do navegador. É invaluable durante desenvolvimento e também para futuros mantenedores entenderem a API sem precisar ler código.

xUnit 2.9.3 (com `xunit.runner.visualstudio 2.8.2` e `Microsoft.NET.Test.Sdk 17.11.1`) é o framework de testes. Cobre sessenta e nove testes unitários nos repositórios, jobs e controllers, usando SQLite em memória (`Data Source=:memory:`) para isolamento entre testes.

NCronJob 4.10.1 foi adicionado mas atualmente não é usado em produção — Hangfire cobre todas as necessidades de agendamento. Permanece como referência para futuras necessidades de jobs mais simples.

2.3 Bibliotecas do Worker (Int2)

O Worker, em `Hardware & Serviço de Background/BiometricAccess.Worker/`, tem dependências enxutas porque sua responsabilidade é específica: falar com hardware T50M e processar eventos de acesso.

Anviz.SDK 2.0.18 é, sem exagero, a biblioteca mais crítica do projeto. Implementa o protocolo OAE (Open Anviz Ethernet) usado pelos dispositivos da família T-series da Anviz. Escrever esse protocolo do zero seria um esforço de meses — são dezenas de comandos binários com framing, checksum, criptografia interna e tratamento de respostas assíncronas. A SDK encapsula tudo em uma API .NET assíncrona. As operações que usamos cobrem o ciclo de vida completo da relação com o T50M: `new AnvizManager().Connect(ip, porta)` abre a sessão TCP/IP na porta 5010; o evento `_device.ReceivedRecord` é assinado para receber push de cada acesso realizado no T50M; `_device.DownloadRecords(onlyNew: true)` drena registros offline armazenados localmente no aparelho (até cinquenta mil) caso o Worker tenha ficado fora do ar; `_device.SetEmployeesData(userInfo)` cadastra ou atualiza um usuário no T50M, definindo ID, nome, senha e modo de verificação (Mode 4 = só senha, Mode 6 = senha ou digital); `_device.DeleteEmployeesData(id)` remove o usuário e seu template biométrico; `_device.SetFingerprintTemplate(id, finger, bytes)` carrega um template biométrico previamente capturado, usado para restaurar biometria sem novo enroll; `_device.GetFingerprintTemplate(id, finger)` baixa o template para backup; `_device.EnrollFingerprint(id)` coloca o T50M em modo enroll para capturar nova digital (bloqueia até a pessoa colocar o dedo duas vezes); e `_device.SetDateTime(DateTime.Now)` sincroniza o relógio do T50M com o servidor, executado diariamente às 03:30 UTC pelo `TimeSyncWorker`.

Microsoft.Extensions.Hosting 8.0.1 e **Microsoft.Extensions.Hosting.WindowsServices 8.0.1** fornecem a infraestrutura de hospedagem para serviços em background. O Worker herda de `BackgroundService` da `Microsoft.Extensions.Hosting`, o que dá um ciclo de vida gerenciado — startup, execução em loop, graceful shutdown — e integração natural com o sistema de injeção de dependências do .NET. A variante `.WindowsServices` permite, com um `UseWindowsService()` no `Program.cs`, que o mesmo executável seja instalado como serviço do Windows com restart automático em caso de falha. No Linux, a integração equivalente é com `systemd` via arquivo de unidade.

2.4 Bibliotecas da infraestrutura (Int4)

O projeto `Int4`, em `InfraestruturaBloco1/`, é uma biblioteca compartilhada (não um serviço) que reúne dois serviços auxiliares.

SharpOnvifClient 0.9.3 implementa o cliente do protocolo ONVIF (Open Network Video Interface Forum), padrão da indústria para interoperabilidade entre sistemas de vídeo e câmeras IP. O `CameraService.MonitorarNovoArquivo()` usa essa biblioteca para confirmar que

uma câmera está respondendo antes de associar a gravação a uma tentativa de acesso. A única função consumida é `client.GetDeviceInformationAsync()` — uma chamada SOAP que retorna informações básicas do dispositivo (fabricante, modelo, firmware). Se a chamada retornar em até cinco segundos, considera-se a câmera operacional e procede-se à construção da URL de Replay no formato Hikvision/Dahua/Intelbras. Se a câmera não responder, o campo `GravacaoPath` da tentativa fica nulo, e o histórico mostra um traço em vez do link.

Microsoft.Extensions.Logging.Abstractions 8.0.2 existe para que o `Int4`, sendo uma biblioteca de classe e não um serviço, possa injetar `ILogger<CameraService>` sem depender de uma implementação concreta de logging. Quem consumir essa biblioteca (no caso, o `Int1`) fornece a implementação (`Serilog`).

2.5 Front-end (Int3)

O `Int3`, em `PainelWeb/Frontend/`, é um projeto **Blazor Server** que faz parte do framework ASP.NET Core 8 e não precisa de pacotes NuGet adicionais para suas funcionalidades core. A única referência externa é o `ProjectReference` ao `Int1` — o que permite ao `Frontend` importar e injetar diretamente os repositórios (`IPessoaRepository`, `IAmbienteRepository` etc.) do backend, sem ter que passar por HTTP para operações de leitura/escrita do banco.

Blazor Server tem características que merecem entendimento. Diferente do Blazor WebAssembly (que executa C# no navegador), Blazor Server mantém o estado e a renderização no servidor; o navegador recebe apenas patches incrementais via SignalR (WebSockets). Cada sessão de admin corresponde a um “circuito” SignalR no servidor, com escopo isolado para serviços Scoped. É por isso que o `ITokenStore` (descrito na seção 6.2) é registrado como Scoped — cada circuito tem sua própria instância, sem vazamento de estado entre administradores logados simultaneamente.

Pacotes adicionais usados no front-end são essencialmente bibliotecas client-side referenciadas via CDN no `App.razor`: **Chart.js** para renderizar o gráfico do dashboard e **HLS.js** para reprodução de streams HLS na visualização ao vivo das câmeras. Ambas são puramente JavaScript, carregadas no navegador, e o Blazor chama suas funções via `IJSRuntime.InvokeVoidAsync(...)`.

3. Arquitetura

3.1 Os quatro projetos

O sistema é composto por quatro projetos .NET, três executáveis e uma biblioteca de classe, com responsabilidades complementares.

O **Int1 — API/Banco** é o coração do sistema. Compilado como um ASP.NET Core Web API, expõe endpoints REST autenticados via JWT, hospeda o arquivo SQLite e executa os jobs agendados via Hangfire. Roda na porta 5018, fixa por convenção (não alterar). Toda a lógica de domínio — repositórios, validações de regra de negócio, criptografia de senhas, geração de PDFs — vive aqui. É o único projeto que serializa o banco em si: nem o painel nem o worker mexem com a conexão SQLite diretamente, mas usam os repositórios do `Int1` importados por referência de projeto.

O **Int2 — Worker** é o serviço de background responsável pelo hardware. Implementado como .NET Worker Service, é o único que dialoga com os dispositivos T50M Anviz via TCP/IP. Mantém um dicionário de conexões abertas (uma por T50M cadastrado), assina os eventos `ReceivedRecord` da SDK, processa cada acesso e grava as tentativas no banco. Roda

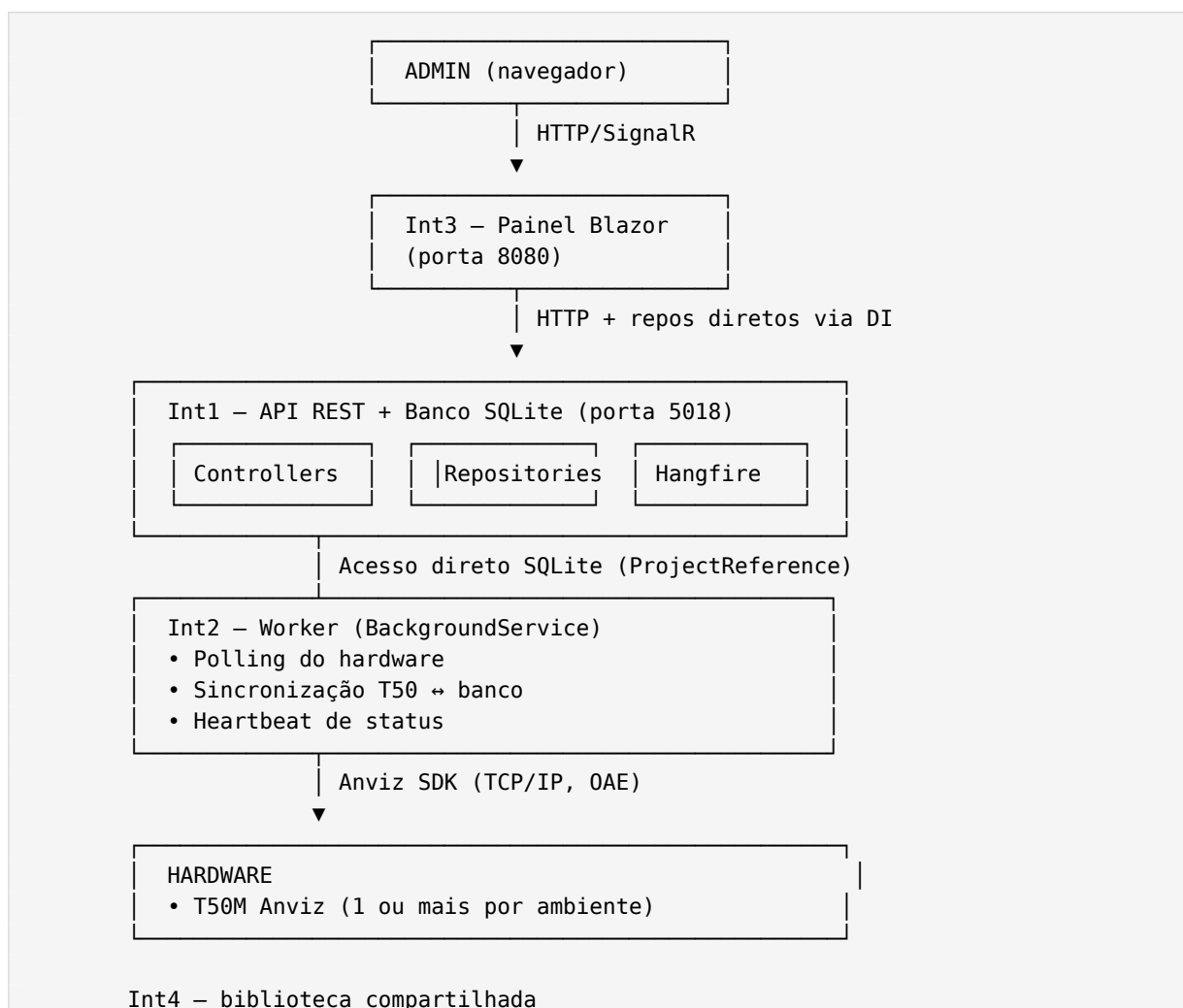
em background sem porta exposta. Quando alguma alteração no painel (como cadastrar uma pessoa em um ambiente) precisa propagar para o hardware, o painel cria registros na tabela T50Pendencia e o SincronizadorT50Worker (dentro do Int2) drena essa fila a cada dez segundos, chamando as operações apropriadas da SDK Anviz.

O **Int3 — Painel** é a interface Blazor Server que os administradores acessam pelo navegador. Roda na porta 8080. Toda a interação humana com o sistema acontece aqui: login, dashboard, cadastros, consultas. Apesar de ter porta HTTP própria, o Int3 só usa essa porta para servir HTML e estabelecer o canal SignalR — todas as operações de banco vão direto pelos repositórios do Int1, sem passar por HTTP. As únicas chamadas HTTP do Int3 para o Int1 são para operações que se beneficiam de processamento centralizado: login (que precisa gerar JWT), exportação de PDF (que pode ser pesada e benefit de cancelamento) e reenvio de email (que precisa decifrar senha AES com a chave do servidor).

O **Int4 — Infraestrutura** é uma biblioteca de classe compartilhada (não tem Program.cs, não é executável). Reúne dois serviços que precisam ser usados tanto pelo Int1 quanto pelo Int2: o CameraService, que faz a integração ONVIF com câmeras IP para buscar URLs de gravação, e o ExportService, que gera CSVs e PDFs. Ambos são chamados primariamente pelo Int1, mas o CameraService é também usado pelo EventProcessor do Int2 quando uma tentativa de acesso precisa de gravação associada.

3.2 Como os projetos se comunicam

O diagrama de comunicação resumido fica assim:



- CameraService (ONVIF), ExportService (CSV/PDF)

A regra mental é: **acesso a banco é por referência de projeto, acesso a hardware é via Worker, comunicação que cruza fronteira de processo é HTTP**. Não há mensageria — nem RabbitMQ, nem Kafka, nem Redis. Para a carga prevista, o overhead de uma camada de mensageria seria injustificado. Quando o painel precisa fazer algo que o Worker vai consumir (como cadastrar uma pessoa em um ambiente), o painel grava a intenção em uma tabela do banco (T50Pendencia) e o Worker drena essa tabela em loop. SQLite atua como mensageria simples e suficiente.

3.3 O modelo de banco compartilhado

O arquivo banco.db reside fisicamente em Banco/WebAbil8-Sistema_Verificação_dupla.slnx /banco.db. Os três projetos que precisam acessá-lo usam caminhos relativos diferentes, todos resolvendo para o mesmo arquivo:

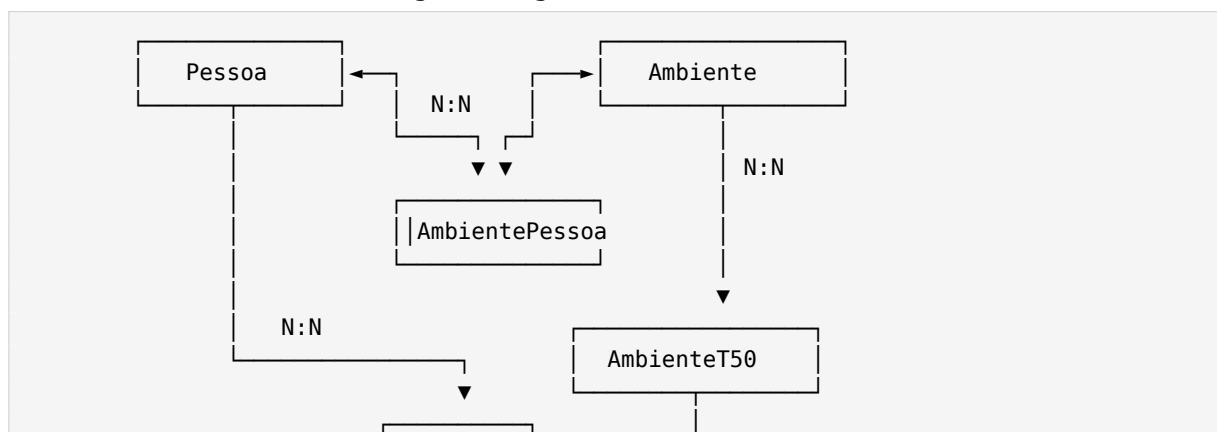
- Int1 (que mora na pasta Banco/.../): caminho relativo direto banco.db.
- Int2 (que mora em Hardware & Serviço de Background/.../): caminho relativo ../../../../Banco/WebAbil8-Sistema_Verificação_dupla.slnx/banco.db.
- Int3 (que mora em PainelWeb/Frontend/): caminho relativo ../../Banco/WebAbil8-Sistema_Verificação_dupla.slnx/banco.db.

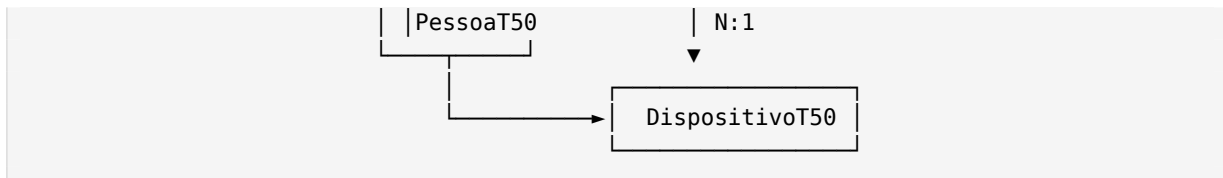
Esses caminhos são definidos nas connection strings dos respectivos appsettings.json. Durante deploy em produção (com os três como serviços), o caminho efetivo depende do WorkingDirectory configurado, então cada serviço é configurado para iniciar na sua pasta publish/ correspondente.

O modo WAL (Write-Ahead Logging) do SQLite é habilitado automaticamente, o que permite que o Int1, o Int2 e o Int3 leiam simultaneamente do banco sem bloquear-se mutuamente. Escritas continuam sequenciais (SQLite é, no fim, um banco com um único arquivo de log), mas para a taxa de escritas do sistema (algumas dezenas por minuto no pior caso) isso não é gargalo. Em uso normal, é comum ver arquivos auxiliares banco.db-shm e banco.db-wal ao lado do banco.db — eles são internos do SQLite e desaparecem quando o banco é fechado limpamente.

4. Banco de Dados

O esquema do banco reflete o domínio de controle de acesso: pessoas que pertencem a ambientes, ambientes que têm dispositivos T50M e câmeras, e um histórico de tentativas e ações administrativas. O diagrama lógico resumido:





4.1 Entidades de negócio

A entidade central é a **Pessoa**, que representa qualquer indivíduo cadastrado para potencialmente ter acesso a algum ambiente. Cada pessoa tem identificadores múltiplos: o Id interno (auto-incremento, usado em chaves estrangeiras), o `CodigoUsuario` (string de seis dígitos do pool, mapeado para o `EmployeeId` que o T50M conhece) e o `Cpf` (único no sistema, validado por dígitos verificadores). Dados básicos incluem `Nome`, `Cargo` (patente ou função), `Email` (para reenvio de credenciais) e `Telefone`. A segurança da senha tem duas representações simultâneas: `senhaHash` (BCrypt fator 10, irreversível, usado para verificação) e `senhaClear` (AES-256-CBC cifrado, reversível, usado quando precisamos reenviar a senha por email). O campo `modoAcesso` é uma string "digital_e_senha" ou "somente_senha", que controla se o sistema exige biometria além do código. O `templateBackup` é um BLOB com o template biométrico exportado do T50M — guardado para que, ao mudar de "somente senha" para "digital e senha" novamente, a biometria seja restaurada sem precisar de enroll novo. O `Status` é "ativo" ou "inativo" e controla se a pessoa pode acessar. `dataUltimoAcesso` é atualizado a cada entrada bem-sucedida e serve de base para a inativação automática por inatividade. `dataCadastro` registra a data de criação.

O **Administrador** representa quem opera o painel. Tem `Id`, `Login` (curto, único), `SenhaHash` (BCrypt) e `NomeCompleto`. Campos opcionais (todos nullable) incluem `Cpf`, `Email`, `Cargo` e `Telefone` — adicionados em migração posterior para enriquecer a auditoria. Crucialmente, **não há campo de status** nem método `Remove()` no repositório: a tabela só pode ser manipulada via banco direto (INSERT, UPDATE, DELETE manuais), conforme decisão de segurança da seção 1.3.

O **Ambiente** representa um espaço físico controlado (sala de operações, armaria, depósito). Tem `Id`, `Nome`, `DispositivoT50Id` (referência ao T50M principal, mantida por compatibilidade com versão anterior), `TempoEsperaGravacaoSeg` (quantos segundos esperar pela câmera disponibilizar a gravação após o acesso, entre 30 e 120), `DataCriacao`, `Excluido` (soft delete) e `DataExclusao`. A relação com T50M e Pessoa é via tabelas de junção (descritas adiante).

O **DispositivoT50** representa um terminal Anviz fisicamente instalado em uma porta. Tem `Id`, `Nome` descritivo (ex: "T50-Entrada-Principal"), `EnderecoIP`, `Porta` (sempre 5010 para Anviz OAE), `DigitaisCadastradas` (contador que indica quantos templates estão ocupados no dispositivo, limite 1000) e `UltimaConexao` (timestamp do último heartbeat com o Worker). A propriedade computada `EstaOnline` retorna verdadeiro se `UltimaConexao` está dentro dos últimos dois minutos.

A **TentativaAcesso** registra cada vez que alguém tentou usar um T50M, com sucesso ou não. Campos: `Id`, `PessoaId` (nullable — pode ser nulo se a pessoa não estava cadastrada), `AmbienteId`, `DataHora` (UTC), `AcessoLiberado` (booleano), `MotivoNegacao` (string com código entre `nao_cadastrado`, `inativo`, `sem_permissao`, `biometria_invalida`, `senha_invalida`, descrito no Apêndice B), `TipoVerificacao` (string indicando o método usado: `digital_id`, `senha_id`, `digital`, `senha`, `primeiro_acesso`), `GravacaoPath` (URL RTSP de Replay da gravação, montada após sucesso no ONVIF) e `DataExpiracao` (data calculada com base em `RetencaoGravacoesTentativasDias` da configuração, usada pelo job de limpeza).

O **LogAdmin** registra cada ação significativa de administrador no painel: login, criação/edição de pessoa, criação de ambiente, alteração de configuração etc. Campos: `Id`, `AdminId`

(FK), Acao (string descritiva: "Login", "Adicionar", "Atualizar", "Remocao", "Inativacao", "ResetBiometria", "ReenvioCredenciais" etc.), EntidadeAfetada (string indicando o tipo: "Pessoa", "Ambiente", "Camera", "Configuracao", "Administrador"), EntidadeId (nullable, indica qual registro foi afetado), DataHora (UTC) e DataExpiracao (calculada a partir de RetencaoLogsDias).

A **Camera** representa uma câmera IP associada a um ambiente. Campos: Id, Nome, AmbienteId, UrlRTSP (obrigatório, para visualização ao vivo via VLC), UrlHLS (opcional, para visualização no navegador via media server intermediário), EnderecoONVIF (opcional, mas necessário para associar gravações), Tipo (string "interna" ou "externa") e Ativa (booleano).

4.2 Tabelas de junção e fila

A **AmbienteT50** é uma junção N:N entre Ambiente e DispositivoT50, permitindo que um ambiente tenha múltiplos T50Ms (por exemplo, portas de entrada e saída de um mesmo espaço) e que um T50M sirva múltiplos ambientes (cenário raro, mas suportado). Tem Id, AmbienteId, DispositivoT50Id, DataVinculo e EhPrincipal (indica qual T50M é o principal de um ambiente — usado quando precisa escolher um único, como na coluna DispositivoT50Id da entidade Ambiente).

A **AmbientePessoa** é a junção N:N que materializa a relação “essa pessoa tem permissão para entrar nesse ambiente”. É a fonte da verdade para autorização: na hora do acesso, o EventProcessor consulta ambientePessoaRepo.PessoaTemAcesso(ambienteId, pessoaId) antes de qualquer outra validação. Sem entrada nessa tabela, a pessoa simplesmente não consegue passar — mesmo que esteja ativa, mesmo com senha correta. Campos: AmbienteId, PessoaId, DataAdicionado.

A **PessoaT50** indica que uma pessoa tem template biométrico cadastrado em um T50M específico. Apenas pessoas com modoAcesso = "digital_e_senha" aparecem aqui. O contador DigitaisCadastradas do DispositivoT50 é mantido em sincronia com o número de entradas nessa tabela. Quando uma pessoa é vinculada a um ambiente, criam-se entradas em PessoaT50 para cada T50M do ambiente, e simultaneamente uma pendência "adicionar" é enfileirada em T50Pendencia para o Worker propagar a alteração ao hardware.

A **T50Pendencia** é a fila de sincronização entre o painel/banco e os dispositivos físicos. Quando o painel faz alguma alteração que precisa chegar ao T50M (cadastrar usuário, remover, restaurar biometria, mudar modo), em vez de chamar o T50M diretamente (o painel não sabe falar com o hardware), o painel grava um registro nessa tabela. O SincronizadorT50Worker no Int2 roda a cada dez segundos, drena até cinquenta pendências por ciclo e executa a ação correspondente via SDK Anviz. Campos: Id, Acao (string entre "adicionar", "remover", "restaurar_biometria", "limpar_biometria"), PessoaId, DispositivoT50Id, CriadoEm, Sincronizado (booleano), SincronizadoEm, TentativasFalhas (contador, sai do round-robin após 5 falhas consecutivas) e ErroUltimaTentativa (mensagem de erro para diagnóstico).

4.3 Tabelas de pools e configuração

A **SenhaDisponivel** é o pool de senhas. Tem Senha (string varchar de 6 caracteres, chave primária), EmUso (booleano) e PessoaId (FK nullable). Seeded no startup do Int1 com 900.000 entradas correspondendo a todos os números de seis dígitos de 100000 a 999999. As triviais (sequências como 123456, 654321, repetições como 111111, e variações simples) são pré-marcadas com EmUso = true para nunca serem distribuídas. Quando uma pessoa é cadastrada, SenhaRepo.BuscarDisponivel() retorna uma senha aleatória dentre as livres, e a transação marca a senha como em uso e a vincula à pessoa.

A **CodigoDisponivel** segue o mesmo padrão para o CodigoUsuario da Pessoa — o ID de

seis dígitos que aparece no T50M. Mesmo pool de 100000 a 999999, mesmo bloqueio de triviais. A separação entre senhas e códigos é proposital: a senha e o código de uma mesma pessoa precisam ser diferentes (caso contrário, qualquer um que conheça o código também conheceria a senha).

A **Configuracao** é uma tabela singleton — sempre tem exatamente uma linha. Campos: `RetencaoGravacoesTentativasDias` (entre 30 e 180, padrão 90 — quantos dias guardar tentativas de acesso), `RetencaoLogsDias` (entre 90 e 365, padrão 180 — quantos dias guardar logs de admin), `TempoEsperaGravacaoSeg` (entre 30 e 120, padrão 60 — segundos para esperar pela gravação da câmera), `PeriodoInativacaoMeses` (entre 3 e 24, padrão 24 — meses sem acesso para inativação automática). A consulta sempre usa `FirstOrDefault()`.

4.4 Migrações e seeding

As migrações ficam em `Banco/WebAbil8-Sistema_Verificação_dupla.slnx/Migrations/` e são geradas via comandos padrão do EF Core a partir do diretório do `Int1`: `dotnet ef migrations add NomeDaMigracao` para criar uma nova migração baseada nas diferenças entre o modelo C# atual e o esquema do banco, e `dotnet ef database update` para aplicá-la.

Adicionalmente, o `Program.cs` do `Int1` faz **migração inline** no startup. Após `db.Database.EnsureCreated()`, executa um `PRAGMA table_info('administrador')` para descobrir quais colunas existem na tabela `Administrador`, e em seguida `ALTER TABLE` apenas para as colunas que estão faltando. Isso garante que bancos de versões antigas (instalados antes de a migração existir) atualizem automaticamente quando a aplicação é reiniciada com a nova versão, sem precisar rodar comandos manuais.

O seeding inicial cria os dados base no primeiro start. Inclui: as 900.000 entradas em `SenhaDisponivel` e em `CodigoDisponivel`, o administrador padrão (login: "admin", senha: "Admin@123") e a linha única em `Configuracao` com os valores padrão. A criação acontece apenas se a tabela correspondente está vazia, então rodar o sistema várias vezes não duplica nada.

5. Backend — Int1 (API)

5.1 Camadas e responsabilidades

O `Int1` segue uma estrutura clássica em camadas que ficou estável ao longo do desenvolvimento.

A camada de **Controllers** (`Banco/.../Controllers/`) recebe requisições HTTP, valida entrada, chama os repositórios e retorna respostas formatadas. Todos os controllers exigem `[Authorize]` exceto o `AuthController` (que faz o login) e o `HealthController` (público para monitoramento externo). Cada controller injeta os repositórios que precisa via construtor — padrão DI nativo do ASP.NET Core.

A camada de **Services / Repositórios** (`Banco/.../Services/`) encapsula o acesso ao banco. Para cada entidade principal há uma interface (`IPessoaRepository`, `IAmbienteRepository` etc.) e uma implementação correspondente em `Services/Implementations/`. As implementações usam o `AppDbContext` (definido em `Model/Context/AppDbContext.cs`) para fazer queries via LINQ. Sempre que uma query envolve navegação para outra entidade (como buscar uma `Pessoa` e acessar seus `Ambientes`), o `.Include(...)` é obrigatório porque o EF Core 8 não tem lazy loading por padrão. Os repositórios são registrados como `Scoped` no `Program.cs`, o que significa uma instância por requisição HTTP no `Int1`, ou uma instância por evento processado no `Worker`.

A camada de **Modelos** (Banco/.../Model/) define as entidades como classes C# puras (PO-COs). Cada propriedade marcada com [Key] é a chave primária; relações navegacionais são propriedades referenciais (public virtual Pessoa Pessoa { get; set; }). O ApplicationDbContext declara um DbSet<T> para cada entidade.

A camada de **Jobs** (Banco/.../Model/Context/) tem as duas classes que rodam pelo Hangfire: InativarUsuariosInativos2AnosJob e LimparDadosExpiradosJob. Cada uma tem um método Executar() que recebe os repositórios via construtor (injeção feita pelo Hangfire).

5.2 Autenticação JWT

O fluxo de login é direto. O painel envia POST /api/auth/login com {Login, Senha}. O AuthController.Login busca o administrador pelo login, compara a senha digitada com o hash BCrypt armazenado via BCrypt.Net.BCrypt.Verify(...), e se confere, gera o JWT.

O JWT é gerado com algoritmo HMAC-SHA256, chave de pelo menos 32 caracteres lida de Jwt:Key no appsettings.json, com claims contendo o adminId e o nomeCompleto. A vida útil do token é de oito horas, o que é confortável para uma jornada de trabalho administrativo. Após gerar, o controller também grava um LogAdmin com ação "Login" para auditoria.

No Program.cs, a autenticação é configurada com builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...), definindo issuer, audience, lifetime válido e a chave de validação. Todos os controllers que devem exigir token recebem [Authorize] na declaração da classe.

5.3 Controllers e endpoints

O **AuthController** (/api/auth) tem apenas o endpoint POST /login.

O **PersonController** (/api/person) cobre operações sobre Pessoa que precisam atravessar HTTP. O principal é POST /{id}/reenviar-credenciais (com alias POST /{id}/reenviar-senha), que descriptografa a senhaClear da pessoa via AES e envia por email usando MailKit. O fluxo completo dessa função está detalhado na seção 12.1. Outros endpoints são CRUD básico — listar, buscar por ID, atualizar e remover (com hard delete e log de auditoria).

O **ExportController** (/api/export) gera relatórios. Os endpoints incluem GET /historico.pdf, GET /historico.csv, GET /logs.pdf, GET /pessoas.csv, GET /admins.pdf e GET /relatorio-ambiente.pdf. Aceitam filtros via query string (?search=...&status=...&de=2026-01-01&ate=2026-12-31). Internamente delegam ao ExportService no Int4, que usa CsvHelper e QuestPDF para gerar os arquivos.

O **HealthController** (/api/health) é o único sem [Authorize] — público intencionalmente para que sistemas externos de monitoramento possam checar saúde do servidor com GET / retornando 200 OK e um JSON com status e uptime.

5.4 Repositórios

Os repositórios são consumidos diretamente pelo Int3 (Painel) via injeção de dependência, sem passar por HTTP. Eis a lista completa:

Interface	Implementação	Função
IPessoaRepository	PessoaImplementations	CRUD Pessoa + cifragem AES da senhaClear
IAmbienteRepository	AmbienteImplementations	CRUD Ambiente

Interface	Implementação	Função
IAmbientePessoaRepository	AmbientePessoaImplemetions	Junction Ambiente↔Pessoa
IAmbienteT50Repository	AmbienteT50Implemetions	Junction Ambiente↔T50 + sync de pessoas
IPessoaT50Repository	PessoaT50Implemetions	Junction Pessoa↔T50 + counter
IDispositivoT50Repository	DispositivoT50Implemetions	CRUD T50 + heartbeat
ICameraRepository	CameraImplemetions	CRUD Camera
ITentativaAcessoRepository	TentativaAcessoImplemetions	CRUD com filtros
ILogAdminRepository	LogAdminImplemetions	Audit log (preenche DataExpiracao)
IConfiguracaoRepository	ConfiguracaoImplemetions	Singleton config
IAdministradorRepository	AdministradorImplemetions	Listar/buscar admins (sem create)
ISenhaRepository	SenhaImplemetions	Pool de senhas
ICodigoRepository	CodigoImplemetions	Pool de códigos
IT50PendenciaRepository	T50PendenciaImplemetions	Fila de sync hardware
IStatusService	StatusServiceImplemetions	Status agregado (dashboard)

Cada repositório segue o mesmo padrão: construtor que recebe o `AppDbContext`, métodos públicos que executam operações específicas, e nenhum estado interno além do contexto. A escopo `Scoped` no DI garante que cada requisição HTTP no `Int1` ou cada evento processado no `Worker` tem seu próprio `AppDbContext` (gerenciamento de transações isolado).

5.5 Jobs agendados (Hangfire)

Dois jobs rodam todos os dias às 03:00 UTC (aproximadamente meia-noite no horário de Brasília), registrados no `Program.cs` do `Int1`:

```
RecurringJob.AddOrUpdate<InativarUsuariosInativos2AnosJob>(
    "inativar-usuarios", j => j.Executar(), "0 3 * * *");

RecurringJob.AddOrUpdate<LimparDadosExpiradosJob>(
    "limpar-dados", j => j.Executar(), "0 3 * * *");
```

O **InativarUsuariosInativos2AnosJob** busca todas as pessoas ativas cujo `dataUltimoAcesso` é mais antigo que `PeriodoInativacaoMeses` (padrão 24) ou nulo (nunca acessou). Para cada uma, remove os vínculos em `AmbientePessoa` e `PessoaT50`, decrementa o `DigitaisCadastradas` dos T50Ms afetados, enfileira pendências "remover" na `T50Pendencia` para cada T50M onde a pessoa estava cadastrada, e marca o status como "inativo". A pessoa não é deletada — só inativada, preservando histórico de tentativas e auditoria.

O **LimparDadosExpiradosJob** apaga registros em três tabelas: `TentativaAcesso` onde `DataExpiracao < DateTime.UtcNow`, `LogAdmin` na mesma condição, e ambientes com `Excluido = true` que não tenham tentativas vinculadas (purga de soft delete antigos). O preenchimento

correto de `DataExpiracao` ao criar cada tentativa e log é crítico para esse job funcionar — historicamente foi um bug (`LogAdmin` com `DataExpiracao` nulo nunca era apagado), corrigido garantindo que `LogAdminRepo.Registrar()` calcule e preencha a data ao criar.

O dashboard do Hangfire em `http://localhost:5018/hangfire` mostra histórico de execuções, falhas, jobs em fila e jobs recorrentes. Está protegido pelo mesmo JWT do resto do sistema.

5.6 Criptografia (AES)

A `senhaClear` da pessoa precisa ser armazenada de forma reversível — afinal, o sistema reenvia a senha por email quando o admin pede, e isso exige recuperar o texto original. Mas armazenar em plaintext seria inaceitável. A solução é cifrar com AES-256-CBC.

A implementação está em `Banco/.../Services/AesHelper.cs`, uma classe estática com três métodos: `Encrypt(string plain, string key)` retorna Base64 de `[IV (16 bytes) || ciphertext]`, `Decrypt(string cipher, string key)` faz o inverso, e `ResolverChave(IConfiguration config)` lê a chave da configuração e ajusta para exatamente 32 bytes (`PadRight(32).Substring(0, 32)`).

O IV (vetor de inicialização) é gerado aleatoriamente a cada chamada de `Encrypt`, garantindo que duas cifragens da mesma senha resultem em ciphertexts diferentes — propriedade conhecida como semantic security. O ciphertext final tem o formato `Base64( IV[16] || EncryptedData )`, e a decifragem separa os primeiros 16 bytes como IV.

A chave AES vem de `configuration["AesKey"]`. Em desenvolvimento, está em `appsettings.json` como `"AesKey": "5cta-aes-key-senha-segura-32char"`. Em produção, deve vir de variável de ambiente para não ficar no repositório de código:

```
export AesKey="<chave segura aqui>"
```

O `AesHelper` é usado em dois pontos: no `PessoaImplementations.Adicionar()` cifrar a `senhaClear` antes de salvar (transparente para os controllers, que passam a senha em plaintext), e no `PersonController.ReenviarCredenciais()` para decifrar a senha cifrada no momento de montar o email.

6. Frontend — Int3 (Painel Blazor)

6.1 Por que Blazor Server

A escolha de Blazor Server sobre alternativas como React, Vue ou Angular foi pragmática. A equipe é primariamente backend, com profundo conhecimento em C# e .NET. Blazor permite escrever a UI inteira na mesma linguagem do backend, reutilizando os mesmos modelos do banco diretamente (não precisa serializar/deserializar entre layers), e compartilhando o ecossistema de bibliotecas. A versão Server (e não WebAssembly) ainda traz a vantagem de manter o estado no servidor, o que simplifica autenticação (o token JWT fica em memória do servidor, não em `localStorage` do navegador exposto a XSS) e permite que a UI tenha acesso direto aos repositórios via DI.

A trade-off conhecida do Blazor Server é a dependência de SignalR ativo — uma conexão WebSocket persistente entre navegador e servidor. Se a conexão cai, a UI fica em estado inválido até reconectar. Em uma rede local do CTA, com latência baixa e poucos usuários simultâneos, isso não é problema. Em cenários de Internet pública com milhares de usuários, Blazor WebAssembly ou um SPA tradicional seriam mais apropriados.

6.2 O ITokenStore e o ciclo de vida do circuito

Cada admin logado no painel corresponde a um circuito SignalR no servidor. O Blazor mantém o estado da UI desse admin (página atual, formulários preenchidos, etc.) por todo o tempo da sessão. Para guardar o JWT e dados do admin, criamos o ITokenStore:

```
public interface ITokenStore
{
    string? Token { get; set; }
    int AdminId { get; set; }
    string NomeCompleto { get; set; }
    bool EstaAutenticado { get; }
}
```

Implementado por TokenStore : ITokenStore, é registrado no Program.cs do Int3 como builder .Services.AddScoped<ITokenStore, TokenStore>(). O escopo Scoped é a parte crítica: cada circuito Blazor tem sua própria instância, sem vazamento de estado entre admins logados simultaneamente. Originalmente o TokenStore era estático (public static class), o que causava o bug de um admin ver os dados do outro — corrigido refatorando para instância scoped.

O CircuitHandler.cs injeta ITokenStore via construtor e implementa OnCircuitClosedAsync para limpar token, adminId e nomeCompleto quando o circuito fecha (típicamente quando o admin fecha o navegador ou perde conexão), evitando que dados persistam após logout.

A verificação de autenticação fica no MainLayout.razor, no método OnAfterRenderAsync(firstRender). Se !TokenStore.EstaAutenticado, redireciona para /login com Nav.NavigateTo("/login"). A escolha por OnAfterRenderAsync em vez de OnInitialized é deliberada e resolve um bug específico: navegação em OnInitialized causa redirect HTTP 302 (full-page load), o que faz o evento JavaScript enhancedload não disparar, e como enhancedload é o que remove o overlay de loading (#app-loading), a tela ficaria preta para sempre. Em OnAfterRenderAsync, a navegação é client-side, o overlay é removido corretamente.

6.3 Páginas e roteamento

As páginas Razor estão em PainelWeb/Frontend/Components/Pages/. Cada arquivo .razor declara uma rota com @page "/caminho" e implementa a UI. As páginas principais incluem:

- Home.razor (rota /) — Dashboard com cards de estatísticas e gráfico Chart.js.
- Login.razor (rota /login) — Formulário de login que faz POST /api/auth/login.
- Pessoas/Index.razor (rota /pessoas) — Lista de pessoas com filtros e botão de cadastrar nova.
- Pessoas/Detalhe.razor (rota /pessoas/{id:long}) — Detalhe de pessoa com ações.
- Ambientes/Index.razor, Ambientes/Detalhe.razor — Lista e detalhe de ambientes.
- DispositivosT50/Index.razor, DispositivosT50/Detalhe.razor — Gerenciamento de T50Ms.
- Cameras.razor — Lista de câmeras e cadastro.
- Historico.razor — Tabela de tentativas com filtros, ordenação e exportação.
- Logs.razor — Tabela de logs de admin.
- Admins/Index.razor, Admins/Detalhe.razor — Visualização de administradores (sem create/delete).
- Configuracoes.razor — Configurações do sistema (retenção, tempo de espera etc.).
- Ajuda.razor — Tela de ajuda com FAQ.

O componente NavMenu.razor define o menu lateral, com links agrupados em “Principal”, “Gerenciamento” e “Sistema”. O MainLayout.razor é o layout padrão que envelopa as páginas

autenticadas, e o `LoginLayout.razor` é o layout sem menu lateral usado só pela tela de login.

6.4 JavaScript interop

Algumas operações no painel exigem chamar JavaScript a partir do C#. O Blazor expõe isso via injeção de `IJSRuntime`. As principais interops estão em `wwwroot/app.js`:

- `removeAppLoading()` — Remove o overlay de loading inicial após primeira renderização.
- `authStorage.save(token, adminId, nome)` e `authStorage.clear()` — Persistem dados de auth em `sessionStorage` (backup adicional ao `ITokenStore` para sobreviver a refreshes de página).
- `downloadCsv(filename, content)` — Cria um Blob e dispara download programático.
- `baixarPdf(url, token)` — Faz fetch autenticado de PDF e baixa.
- `renderDashboardChart(canvasId, permitidos, negados)` — Inicializa `Chart.js` no canvas do dashboard.
- `updateDashboardChart(permitidos, negados)` — Atualiza os dados sem recriar o chart (mais performático).
- `maskCpf(input)`, `maskTelefone(input)`, `maskTextoSeguro(input, max)` — Máscaras de input.
- Funções de player HLS para visualização ao vivo das câmeras.

A comunicação é unidirecional: o C# chama JS, mas JS não chama C# diretamente. Quando precisamos de callback do JS para o C# (raramente), usamos `[JSInvokable]` em métodos C# expostos como invocáveis.

7. Worker — Int2 (Serviço de Background)

7.1 BackgroundService principal

A classe `Worker.cs` herda de `BackgroundService` da `Microsoft.Extensions.Hosting` e implementa o método `ExecuteAsync` com o loop principal. Em pseudocódigo simplificado:

```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        if (!_connector.Conectar()) { await Task.Delay(10s); continue; }
        RegistrarHeartbeat();
        var eventosArmazenados = _connector.BuscarEventosArmazenados();
        foreach (var ev in eventosArmazenados) await _eventProcessor.Processar(ev);

        while (!stoppingToken.IsCancellationRequested) {
            var evento = _connector.BuscarNovoEvento();
            if (evento != null) await _eventProcessor.Processar(evento);
            if (--ciclos <= 0) { RegistrarHeartbeat(); ciclos = 30; }
            await Task.Delay(2s);
        }
    }
}
```

O loop externo gerencia reconexão: se `Conectar()` falhar, espera dez segundos e tenta de novo. Após conectar, registra um heartbeat (atualiza `UltimaConexao` do `DispositivoT50` no banco para refletir que está online), drena registros offline acumulados durante o tempo em que esteve desconectado (`BuscarEventosArmazenados()` chama `DownloadRecords(onlyNew: true)` da SDK), e entra no loop interno de polling.

O loop interno verifica se há novo evento na fila local do connector a cada dois segundos. Como a SDK Anviz já expõe eventos via push (callback `ReceivedRecord`), o que o loop faz na verdade é consumir a `ConcurrentQueue` que o connector preenche em background quando o callback dispara. A cada trinta ciclos (aproximadamente um minuto), registra outro heartbeat para manter o status online no painel.

7.2 Camada de conexão (AnvizConnector)

A interface `IAnvizConnector` define o contrato:

```
public interface IAnvizConnector
{
    string EnderecoIdentificador { get; }
    bool Conectar();
    EventoAcesso? BuscarNovoEvento();
    List<EventoAcesso> BuscarEventosArmazenados();
    void Desconectar();
}
```

A implementação `AnvizConnector` (em `Services/AnvizConnector.cs`) encapsula a SDK Anviz. No `Conectar()`, instancia `new AnvizManager().Connect(_ip, _porta).Result`, recebe o `AnvizDevice` resultante e assina os eventos `ReceivedRecord` e `DeviceError`. O handler `OnReceivedRecord` converte cada `Record` da SDK em um `EventoAcesso` (modelo interno do Worker), inferindo `TipoVerificacao` a partir do `BackupCode` do registro (`BackupCode == 4` significa senha, qualquer outro valor é digital) e inferindo `AcessoLiberado` do bit 7 do `RecordType` (1 = porta abriu, 0 = não abriu).

Os eventos são enfileirados em uma `ConcurrentQueue<EventoAcesso>` interna, e `BuscarNovoEvento()` simplesmente faz `_filaEventos.TryDequeue(out var evento)` — desacoplando a recepção (que acontece em thread do SDK) do processamento (que acontece no loop do Worker).

7.3 Camada de processamento (EventProcessor)

A interface `IEventProcessor` tem um único método: `Task Processar(EventoAcesso evento)`. A implementação `EventProcessor` recebe um `IServiceScopeFactory` no construtor — fundamental porque o `EventProcessor` é registrado como Singleton (existe um por T50M), mas os repositórios são Scoped (precisam de novo escopo a cada operação). O padrão correto, evitando o anti-pattern “captive dependency”, é criar um scope por evento:

```
public async Task Processar(EventoAcesso evento)
{
    using var scope = _scopeFactory.CreateScope();
    var sp = scope.ServiceProvider;
    var pessoaRepo = sp.GetRequiredService<IPessoaRepository>();
    var ambientePessoaRepo = sp.GetRequiredService<IAmbientePessoaRepository>();
    // ...
}
```

O fluxo de processamento segue uma sequência fixa. Primeiro encontra o `DispositivoT50` pelo `IpDispositivo` do evento. Depois resolve o Ambiente correspondente — via tabela `AmbienteT50` quando há múltiplos vínculos, ou direto via `DispositivoT50Id` do ambiente principal. Registra heartbeat (`UltimaConexao = UtcNow`). Busca a Pessoa por `CodigoUsuario` (com fallback para `Id` por compatibilidade com registros antigos). Valida em cascata: a pessoa existe? Está ativa? Tem permissão no ambiente (`PessoaTemAcesso(ambienteId, pessoaId)`)? Roteia pelo `TipoVerificacao`. Registra uma `TentativaAcesso` com `DataExpiracao` calculada a

partir da configuração de retenção. Se o acesso foi liberado, agenda em fire-and-forget (`Task.Run(...)`) a chamada ao `CameraService.MonitorarNovoArquivo()` para tentar capturar a URL de gravação após o tempo de espera configurado.

7.4 Sincronizadores periódicos

Além do Worker principal, o Int2 hospeda dois `BackgroundServices` adicionais.

O **SincronizadorT50Worker** roda a cada dez segundos. Drena `T50Pendencia.ListarPendentes(50)` e processa cada pendência conforme sua `Acao`:

```
bool ok = p.Acao switch
{
    "adicionar" => _anvizService.AdicionarPessoa(codigoT50, nome, senhaPlain),
    "remover"    => _anvizService.RemoverPessoa(codigoT50),
    "restaurar_biometria" => RestaurarBiometria(codigoT50, pessoa, senhaPlain),
    "limpar_biometria"    => _anvizService.AlterarModo(codigoT50, "senha"),
    _ => false
};
```

`RestaurarBiometria` chama `AdicionarPessoa` (idempotente, então pode rodar várias vezes sem dano), seguido de `UploadTemplate(templateBackup)` com o template guardado no banco, e finalmente `AlterarModo("ambos")` (Mode 6 = FP|PWD bitmask). Pendências que falham cinco vezes consecutivas saem do round-robin e precisam intervenção manual via `T50Pendencia.MarcarResultado/Falha`.

O **TimeSyncWorker** sincroniza a hora dos T50Ms com a hora do servidor, diariamente às 03:30 UTC (trinta minutos depois dos jobs do Int1, para dar margem entre limpezas e sync de hora). Chama `_anvizService.SincronizarHora()` para cada T50M conectado, que internamente faz `_device.SetDateTime(DateTime.Now)`.

8. Infraestrutura — Int4

8.1 CameraService e integração ONVIF

O `CameraService` em `InfraestruturaBloco1/Services/CameraService.cs` é o único contato do sistema com câmeras IP. Tem duas operações públicas: `ObterUrlStream(int cameraId)` retorna a URL RTSP cadastrada para visualização ao vivo, e `MonitorarNovoArquivo(int ambienteId, DateTime timestamp, int tempoEsperaSeg)` faz o trabalho mais complexo de associar uma gravação a uma tentativa de acesso.

A documentação técnica original do projeto (`doc_tecnica.docx` §5.11) define que a câmera grava sozinha quando detecta movimento; o sistema não controla a gravação. Após um acesso liberado, queremos apenas pegar o link para o trecho gravado. O fluxo do `MonitorarNovoArquivo` é o seguinte:

```
public async Task<string?> MonitorarNovoArquivo(int ambienteId, DateTime timestamp, int
    tempoEsperaSeg)
{
    // 1. Aguarda tempoEsperaSeg (30-120s) pra câmera capturar
    // 2. Conecta na câmera via SimpleOnvifClient (ONVIF)
    // 3. Verifica que a câmera responde (GetDeviceInformation)
    // 4. Monta URL de Replay RTSP no formato Hikvision/Dahua/Intelbras
    // 5. Retorna a URL pra ser persistida em TentativaAcesso.GravacaoPath
}
```

Primeiro busca todas as câmeras do ambiente. Prefere câmeras com Tipo = "externa" e EnderecoONVIF preenchido (porque externas geralmente capturam o pátio e a entrada, mais útil do que internas), e cai em câmeras internas se nenhuma externa estiver disponível. Aguarda o tempo configurado (entre 30 e 120 segundos) para dar tempo da câmera processar e disponibilizar o trecho gravado em seu próprio storage. Em seguida tenta conectar no EnderecoONVIF com `new SimpleOnvifClient(enderecoOnvif, usuario, senha)` e chama `await client.GetDeviceInformationAsync().WaitAsync(cts.Token)` com timeout de cinco segundos. Se essa chamada for bem-sucedida, a câmera está online e as credenciais ONVIF são válidas. Caso contrário, a função retorna null e a tentativa fica sem gravação associada (— no histórico do painel).

Confirmada a resposta ONVIF, a função monta uma URL RTSP de Replay no formato:

```
rtsp://<user>:<pass>@<host>:<port>/Streaming/tracks/101?starttime=<UTC>&endtime=<UTC>
```

Esse formato é padrão Hikvision, mas Dahua e Intelbras adotaram um formato compatível, então funciona para a maioria das câmeras profissionais. As credenciais e host são extraídas da `UrlRTSP` cadastrada na câmera. O `starttime` é o timestamp do acesso, e o `endtime` é cinco minutos depois — janela ampla o suficiente para capturar a chegada e a entrada da pessoa.

A escolha de implementar o Profile G completo do ONVIF (com `FindRecordings` + `GetReplayUri`, que seriam SOAP) foi descartada porque exige tratamento específico por fabricante e a versão da `SharpOnvifClient` usada não expõe esses métodos de forma confiável para todas as marcas. Para câmeras que não suportem o formato de Replay padrão, `GravacaoPath` fica nulo — o sistema continua funcionando, só não exibe link para a gravação.

8.2 ExportService (CSV e PDF)

O `ExportService`, também em `InfraestruturaBloco1/Services/`, gera relatórios em CSV e PDF. Para CSV usa **CsvHelper**, escrevendo um stream de memória com headers e linhas de dados. Para PDF usa **QuestPDF**, com sua API fluente declarativa. Os relatórios disponíveis incluem histórico de acessos com filtros, lista de logs de admin, lista de pessoas com filtros, lista de admins e relatório completo de ambiente (com pessoas vinculadas, T50Ms e tentativas recentes).

O `ExportController` do `Int1` expõe esses serviços via HTTP, aplica os filtros vindos da query string e devolve o arquivo com o Content-Type correto (`text/csv` ou `application/pdf`). Como a geração de PDF pode ser pesada para relatórios grandes, o cliente (painel) faz a chamada com timeout maior e mostra spinner durante o download.

9. Fluxos de Negócio

9.1 Cadastro de pessoa

Iniciado pelo admin na tela `Pessoas/Index.razor`. O fluxo completo segue uma sequência de chamadas que combinam várias regras de negócio.

O `HandleCadastrar()` valida primeiro os campos (nome, email, CPF, cargo) e usa helpers (`FormatHelper.CpfValido`, `FormatHelper.EmailValido`) para sanitização. Em seguida chama `CodigoRepo.BuscarDisponivel()` para obter um código de seis dígitos livre do pool, e `SenhaRepo.BuscarDisponivel()` para obter uma senha. Há um loop que regeira a senha até cinco vezes

se ela coincidir com o código — não pode haver pessoa cuja senha seja igual ao seu próprio código.

Cria o objeto Pessoa em memória com Status = "inativo" (será ativada quando vinculada a ambiente), e chama PessoaRepo.Adicionar(pessoa). O repositório, por sua vez, valida CPF único e chama AesHelper.Encrypt(senhaClear) para cifrar a senha antes de persistir. O hash BCrypt já foi gerado em HandleCadastrar() com BCrypt.Net.BCrypt.HashPassword(senha, workFactor: 10).

Após o SaveChangesAsync, marca o código e a senha como em uso (CodigoRepo.MarcarEmUso(...) e SenhaRepo.Atualizar(...)), registra um LogAdmin com ação "Adicionar", e enfim mostra ao admin o ID e a senha gerados em uma modal.

Imediatamente após exibir a modal, o frontend chama EnviarCredenciaisAsync(pessoa.Id) que faz POST /api/person/{id}/reenviar-credenciais para o Int1. O endpoint do Int1 descryptografa a senha via AES e envia o email com ID e senha usando MailKit. O frontend atualiza o status visual da modal para indicar se o email foi enviado, está enviando, ou falhou (caso o SMTP não esteja configurado).

9.2 Vínculo de pessoa a ambiente

A pessoa cadastrada está com Status = "inativo" e precisa ser vinculada a pelo menos um ambiente para acessá-lo. Isso acontece na tela Ambientes/Detalhe.razor quando o admin clica em "Adicionar Pessoa".

O HandleAdicionarPessoa() faz primeiro um pre-check de capacidade: para cada T50M do ambiente, verifica se DigitaisCadastradas < 1000. Se algum estiver lotado e a pessoa é digital_e_senha, o vínculo é bloqueado. Caso contrário, prossegue.

Chama AmbientePessoaRepo.AdicionarPessoa(...) para criar a entrada na tabela de junção. Chama PessoaRepo.AlterarStatus(pessoa, ativo: true) para reativar. Se a pessoa é digital_e_senha, para cada T50M do ambiente: chama PessoaT50Repo.Adicionar(pessoaId, t50Id) (que incrementa o contador DigitaisCadastradas) e T50PendenciaRepo.Enfileirar("adicionar", pessoaId, t50Id).

Há um caso especial: se a pessoa já tinha templateBackup no banco (de cadastro anterior) e biometriaCadastrada é null, marca biometriaCadastrada =.UtcNow localmente — isso indica restauração rápida via template guardado, sem precisar de enroll novo. A pendência "adicionar" que o Sincronizador vai processar inclui o upload do template via SetFingerprintTemplate.

9.3 Primeiro acesso e enroll biométrico

Pessoa com modoAcesso = "digital_e_senha" e biometriaCadastrada = null está no estado de primeiro acesso. O fluxo no T50M é o seguinte:

A pessoa digita o ID no teclado do T50M. A SDK Anviz entrega o evento via _device.ReceivedRecord. O EventProcessor recebe, encontra a pessoa, valida que está ativa e tem permissão. Vê que biometriaCadastrada == null e que o modo é digital_e_senha. Notifica o T50M para pedir senha (com flag de "primeiro acesso", o que pode mostrar mensagem específica no display do dispositivo).

A pessoa digita a senha. A SDK entrega outro evento. O EventProcessor decifra a pessoa.senhaClear via AES e compara com a senha digitada. Se confere, entra no fluxo de enroll: notifica o T50M para entrar em modo EnrollFingerprint. A pessoa coloca o dedo duas vezes no leitor. O T50M captura, gera o template e dispara um evento de sucesso. O EventProcessor marca pessoa.biometriaCadastrada =.UtcNow, opcionalmente baixa o template para

guardar como `templateBackup`, e libera o acesso (a porta abre).

A partir daí, próximos acessos da mesma pessoa no mesmo T50M usam o fluxo normal de digital.

9.4 Acesso normal por digital

A pessoa digita o ID e em seguida coloca o dedo no leitor. O T50M faz o match localmente contra os 1000 templates armazenados. Se bate, envia evento com `BackupCode != 4` (digital) e `RecordType` com bit 7 ligado (acesso liberado). Se não bate, envia evento com bit 7 desligado e `BackupCode` indicando falha biométrica.

O `EventProcessor` processa o evento conforme o resultado. Em caso de sucesso, registra `TentativaAcesso(AcessoLiberado: true, TipoVerificacao: "digital")`, atualiza `pessoa.dataUltimoAcesso =.UtcNow`, e agenda fire-and-forget a chamada ao `CameraService.MonitorarNovoArquivo` em `Task.Run`. Como o T50M já liberou o relé localmente (Lock Relay Time de 15 segundos), o processamento do C# é puramente de registro e auditoria — a porta já está aberta antes mesmo do processamento terminar.

Em caso de falha, registra a tentativa como negada com `MotivoNegacao = "biometria_invalida"`.

9.5 Acesso normal por senha

Idêntico ao primeiro acesso até a digitação da senha. A diferença é que a partir do passo 5 (sucesso na validação da senha):

- Se `modoAcesso == "somente_senha"` ou `biometriaCadastrada != null` (em qualquer modo): libera direto, sem enroll. `TipoVerificacao = "senha_id"` ou `"senha"`.
- Se `modoAcesso == "digital_e_senha"` e `biometriaCadastrada == null`: cai no fluxo de primeiro acesso descrito na seção 9.3.

9.6 Acesso negado

Para cada acesso negado, o sistema registra uma `TentativaAcesso` com `AcessoLiberado = false` e o motivo apropriado:

Motivo	Quando
<code>nao_cadastrado</code>	ID não existe no banco
<code>inativo</code>	Pessoa existe mas <code>Status = "inativo"</code>
<code>sem_permissao</code>	Pessoa não tem <code>AmbientePessoa</code> no ambiente
<code>biometria_invalida</code>	Match de digital falhou
<code>senha_invalida</code>	Senha digitada não confere
<code>biometria_nao_cadastrada_neste_t50</code>	Modo <code>digital_id</code> mas pessoa não tem <code>PessoaT50</code> nesse T50
<code>t50_lotado</code>	T50M atingiu 1000/1000 templates

A captura desses motivos é importante para auditoria e diagnóstico. Por exemplo, um spike de `nao_cadastrado` em um ambiente pode indicar tentativas de invasão; um padrão de `senha_invalida` por uma pessoa específica pode indicar que ela está digitando errado e precisa de assistência.

9.7 Alteração de modo de acesso

Ação iniciada em Pessoas/Detalhe.razor → HandleAlterarModo. Tem dois sentidos.

Digital + Senha → Somente Senha: para cada T50M onde a pessoa estava cadastrada, chama PessoaT50.Remove (decrementa DigitaisCadastradas) e enfileira pendência "limpar_biometria". Marca biometriaCadastrada = null no banco. Preserva intencionalmente o templateBackup para permitir restauração rápida caso o admin volte o modo no futuro. A pendência "limpar_biometria" no Sincronizador chama _anvizService.AlterarModo(codigoT50, "senha") — Mode 4 = PWD only no T50M.

Somente Senha → Digital + Senha: pre-check capacidade (cada T50M do ambiente tem espaço para mais um template). Para cada T50M do ambiente, chama PessoaT50.Adicionar (incrementa contador) e enfileira pendência. Se templateBackup existe no banco, enfileira "restaurar_biometria" e marca biometriaCadastrada = UtcNow localmente (restauração instantânea após sync). Se não tem template, enfileira "adicionar" e deixa biometriaCadastrada = null (próximo acesso vai ser primeiro acesso, com enroll).

9.8 Inativação automática e limpeza periódica

Ambos rodam às 03:00 UTC via Hangfire, conforme descrito na seção 5.5. O importante para entender o domínio é que essas operações são **idempotentes e seguras**. Apagar uma tentativa expirada é destrutivo, mas a DataExpiracao é calculada com base em uma janela longa (90+ dias por padrão) e auditável — qualquer alteração na retenção é registrada como LogAdmin. Inativar uma pessoa por inatividade não é destrutivo: o registro continua no banco, só com Status = "inativo", então auditoria histórica permanece intacta.

10. Hardware T50M Anviz

10.1 Especificações

Conforme datasheet oficial Anviz:

Parâmetro	Valor
Capacidade de templates biométricos	1.000
Capacidade de registros de acesso (offline)	50.000
Comunicação	TCP/IP (porta 5010)
Protocolo	OAE (Open Anviz Ethernet)
FAR (False Accept Rate)	0,00001%
FRR (False Reject Rate)	0,001%
Tempo de verificação	≤ 1s
Lock Relay Time	15s fixo (configurável no menu local do T50M)
Modos de verificação	Digital+ID (Mode 6), Senha+ID (Mode 4), ambos (Mode 6 = FP PWD bitmask)
Alimentação	12V DC, < 130mA
Interfaces	Ethernet, Wiegand, USB (Pen drive pra backup local)
Temperatura de operação	-10°C a +60°C

Os números de FAR e FRR são notáveis. FAR de 0,00001% significa que a chance de o leitor aceitar um dedo errado como se fosse o certo é de um em dez milhões — muito abaixo do nível de erro humano. FRR de 0,001% é a chance de rejeitar o dedo certo (resultando em frustração para o usuário, mas sem comprometer segurança). Esses números são típicos de sensores ópticos de qualidade industrial.

10.2 Protocolo OAE

OAE (Open Anviz Ethernet) é o protocolo binário proprietário da Anviz para a família T-series. Roda sobre TCP/IP na porta 5010. Estabelece uma sessão persistente onde comandos vão do cliente (Worker) ao dispositivo, e respostas mais eventos espontâneos (registros de acesso) vão do dispositivo ao cliente.

A SDK Anviz.SDK 2.0.18 implementa esse protocolo. Não consumimos OAE diretamente — todas as operações são chamadas de alto nível na SDK. As principais operações usadas estão tabuladas:

Operação	Método SDK	Quando usamos
Conexão	<code>AnvizManager.Connect(ip, port)</code>	Startup do Worker + retry no BackgroundService
Leitura de eventos novos	<code>_device.ReceivedRecord (callback)</code>	Loop principal do Worker (2s polling)
Drenar registros offline	<code>_device.DownloadAllNewRecords()</code>	Reconexão depois de queda de rede
Cadastrar pessoa	<code>_device.SetEmployeesData(userInfo)</code>	Pendência adicionar
Remover pessoa	<code>_device.DeleteEmployee(id)</code>	Pendência remover
Alterar modo (FP/PWD)	<code>_device.SetEmployeesData (Mode 4 ou 6)</code>	Mudança de modo de acesso
Upload template	<code>_device.SetFingerprintTemplate(id, template)</code>	Restauração de biometria
Sincronizar hora	<code>_device.SetDeviceTime(DateTime.Now)</code>	TimeSyncWorker (03:30 UTC)

O Worker mantém uma conexão por T50M em um dicionário `Dictionary<int, IAnvizConnector>` indexado pelo `DispositivoT50Id`. Reconexão é automática a cada dez segundos se `Conectar()` falhar. Eventos vindos do callback `ReceivedRecord` são serializados em uma fila concorrente local antes de serem processados, garantindo que threading da SDK não interfira com o loop do BackgroundService.

Há uma ressalva técnica conhecida sobre senha: o T50M usa um formato especial de 3 bytes para senha numérica (12 bits para o comprimento, 20 bits para o valor). A SDK .NET pode ou não fazer essa conversão internamente — em testes manuais funcionou, mas se autenticação por senha falhar com hardware real em produção, o ponto a investigar primeiro é a linha `userInfo.Password = ulong.Parse(senha)` em `AnvizService.cs:26`. A conversão manual, se necessária, seria deslocar bits do comprimento para os bits 23-20 e o valor numérico para os bits 19-0.

10.3 Operação local no dispositivo

Para cadastros emergenciais (sem painel web) ou primeiro acesso, o T50M tem menu local acessível com botão M/OK:

Operação	Caminho no menu
Cadastrar usuário	M/OK → User → Add → digite ID → coloque dedo (3x) ou senha
Apagar usuário	M/OK → User → Delete → digite ID
Listar usuários	M/OK → User → All Users
Configurar IP	M/OK → System → Comm → Ethernet → IP/Mask/Gateway
Ajustar relógio	M/OK → System → Time → Date/Time
Modo de verificação	M/OK → System → Verify → FP / PWD / FP+PWD
Backup via USB	Inserir pen drive → M/OK → USB Disk → Download

Alterações feitas pelo menu local **não são propagadas** ao banco do nosso sistema. A próxima sincronização do SincronizadorT50Worker pode sobrescrever cadastros locais (a pendência "adicionar" reenviará dados do banco para o T50M, sobrescrevendo o que foi cadastrado manualmente). Por isso, sempre que possível, é melhor usar a UI do painel para qualquer operação rotineira.

11. Segurança

11.1 Estratégia em camadas

A segurança do sistema é construída em camadas, cada uma protegendo contra um tipo diferente de ameaça. Listadas das mais externas às mais internas:

A **rede local** é o primeiro perímetro. O sistema só roda dentro da rede do CTA, atrás do firewall corporativo. Nenhum endpoint do sistema é exposto à Internet. Isso elimina toda uma classe de ataques (scans automatizados, força bruta de credenciais via Internet) por construção arquitetural.

A **autenticação JWT** é a segunda camada. Mesmo dentro da rede, ninguém acessa endpoints da API sem token válido. Tokens expiram em oito horas, forçando re-autenticação periódica. A chave de assinatura JWT vive em variável de ambiente em produção, nunca no código.

O **controle de acesso por administrador** é a terceira. Cada admin tem login único, senha hash BCrypt, e cada ação relevante (criar pessoa, alterar configuração) é logada com adminId em LogAdmin. Auditoria completa do que cada um fez, quando e em qual entidade.

A **criptografia de dados sensíveis** é a quarta. Senhas dos administradores são BCrypt (irreversível); senhas das pessoas são duplas — BCrypt para verificação no T50M e AES-256-CBC para reenvio por email. CPFs ficam em plaintext mas não são expostos via API pública (só via endpoints autenticados).

A **gestão de admins via banco** é a quinta. Como descrito na seção 11.4, criar/remover admin não pode ser feito pela UI. Isso impede que um admin comprometido escale privilégios.

11.2 Armazenamento de senhas

Dado	Algoritmo	Onde
Senha do admin	BCrypt fator 10 (irreversível)	Administrador. SenhaHash
Senha da pessoa (verificação)	BCrypt fator 10	Pessoa.senhaHash
Senha da pessoa (reenvio)	AES-256-CBC	Pessoa.senhaClear

O AES-256-CBC usa chave de 32 bytes lida do `appsettings.json` (campo `AesKey`) em desenvolvimento, ou de variável de ambiente em produção. O IV (vetor de inicialização) é aleatório a cada `Encrypt`, prepended ao ciphertext. Output final em Base64.

A implementação fica em `Banco/.../Services/AesHelper.cs`, conforme detalhado na seção 5.6.

11.3 Autorização nas APIs

Todos os controllers do `Int1` carregam `[Authorize]` na declaração da classe, com a única exceção do `HealthController` que é público intencionalmente para monitoramento.

O JWT é configurado via `Microsoft.AspNetCore.Authentication.JwtBearer` com:

- Algoritmo: HMAC-SHA256
- Chave: `Jwt:Key` (mínimo 32 caracteres)
- Issuer: `Jwt:Issuer`
- Audience: `Jwt:Audience`
- Expira em 8 horas

A validação acontece automaticamente pelo middleware do ASP.NET Core — qualquer requisição com token ausente, expirado, mal assinado ou com claims inconsistentes é rejeitada com 401 antes mesmo de chegar ao controller.

11.4 Gestão de administradores

Decisão arquitetural: administradores só podem ser criados via INSERT direto no banco. A justificativa em três pontos:

1. **Sistema usado em ambiente militar (5º CTA)** — alto risco de comprometimento se a UI permitir CRUD de admins.
2. **Quem tem acesso ao banco já tem acesso ao servidor** — permitir CRUD pela UI seria escalada de privilégio (admin comprometido cria outro admin).
3. **Logs de auditoria são preservados** mesmo após DELETE do admin (FK não cascateia para `LogAdmin`).

O procedimento de criação está documentado em `docs/scripts/README.md` e também na seção 7 do `DeploymentGuide.html`. Resumo: gerar hash BCrypt da senha nova com o utilitário `docs/scripts/GerarHash`, abrir o banco no DBeaver, executar INSERT, fazer commit explícito.

Mudança de senha segue o mesmo padrão: gerar novo hash, UPDATE. Remoção: DELETE, ciente de que logs antigos ficam com `adminId` órfão (intencional — auditoria preservada, exibido como “Admin {id}” no painel).

12. Integrações Externas

12.1 Email via servidor SMTP

A integração de email é deliberadamente minimalista. O sistema não tem servidor de email embutido — é apenas um cliente SMTP que se conecta ao servidor da unidade.

A configuração vem de quatro variáveis de ambiente:

- SMTP_HOST — host do servidor (ex: smtp.ebmail.eb.mil.br)
- SMTP_PORT — porta (587 com STARTTLS é padrão)
- SMTP_USER — usuário SMTP
- SMTP_PASS — senha SMTP

Se qualquer variável estiver ausente, o sistema entra em “fallback console” — escreve no log do servidor as credenciais que deveriam ter sido enviadas, e retorna `fallback: true` na resposta da API. Isso permite operação em ambiente sem SMTP (durante testes, antes de a TI configurar o servidor de email, etc.), com a entrega manual de credenciais a cargo do admin que está olhando os logs.

O endpoint é `POST /api/person/{id}/reenviar-credenciais` no `PersonController.cs`. O HTML do email é uma string literal nas linhas 182-189 do controller — texto HTML simples com tabela do ID e da senha, formatação básica. Para customizar (logo do CTA, cores, mensagem), basta editar essa string e recompilar.

O envio acontece automaticamente após cada cadastro de pessoa: o frontend chama esse endpoint logo depois de criar a pessoa, e atualiza a UI com o status (verde, amarelo, azul) conforme o resultado. O admin também pode disparar o reenvio manualmente em qualquer momento via botão na tela de detalhe da pessoa.

Veja docs/DeploymentGuide.html seção 7 para configuração completa do SMTP em produção, incluindo exemplo para Windows (variáveis de ambiente persistentes) e Linux (arquivo de unidade systemd).

12.2 Câmeras IP via ONVIF e RTSP

O sistema interage com câmeras IP em três frentes técnicas: ONVIF para descoberta e validação, RTSP para a URL base (tanto de visualização ao vivo quanto de Replay), e HLS opcionalmente para streaming no navegador.

ONVIF (Open Network Video Interface Forum) é o padrão da indústria implementado por praticamente toda câmera IP profissional. Usamos a biblioteca **SharpOnvifClient 0.9.3** apenas para uma operação: `client.GetDeviceInformationAsync()`. Essa chamada SOAP retorna informações básicas da câmera (fabricante, modelo, firmware). Se a chamada retornar dentro de cinco segundos, considera-se a câmera operacional e procede-se à associação da gravação. É um ping ONVIF, essencialmente — confirma que a câmera está respondendo e que as credenciais embutidas na URL RTSP são válidas.

RTSP (Real Time Streaming Protocol) é o protocolo padrão das câmeras IP para entrega de vídeo. URLs típicas têm formato `rtsp://usuario:senha@ip:554/caminho-do-stream`. No nosso sistema, a URL RTSP cadastrada é usada de duas maneiras: como endereço para o admin abrir no VLC e ver a câmera ao vivo (player desktop), e como base para construir a URL de Replay que aparece no histórico — só substitui o path por `/Streaming/tracks/101?starttime=...&endtime=...` e mantém as mesmas credenciais.

HLS (HTTP Live Streaming) é o protocolo de streaming sobre HTTP que roda em navegadores modernos. Câmeras IP não falam HLS nativamente — só RTSP. Para ter visualização

ao vivo direta no painel (sem precisar abrir VLC), é necessário um servidor de mídia intermediário (MediaMTX, FFmpeg, nginx-rtmp) que pega o RTSP da câmera e retransmite como HLS. Como nem todo cliente vai querer instalar esse intermediário, o campo HLS é opcional. Detalhes de configuração estão no DeploymentGuide.html seção 6.4.

O sistema funciona com qualquer câmera que suporte ONVIF Profile S e disponibilize stream RTSP. Isso cobre marcas profissionais como Intelbras, Hikvision, Dahua, Axis e Bosch. Câmeras analógicas sem rede e câmeras Wi-Fi residenciais (Wyze, Tapo) geralmente não funcionam.

A limitação principal do sistema é que **não controla a gravação**. A câmera grava sozinha (por movimento ou contínuo), e o sistema apenas guarda o link para encontrar o trecho relevante. Se a câmera não estiver gravando, não há vídeo associado. O Profile G completo do ONVIF (com FindRecordings + GetReplayUri) não foi implementado — o sistema usa a abordagem “montar URL no formato Hikvision/Dahua/Intelbras”, que funciona para essas marcas mas pode não funcionar para outras.

13. Testes Automatizados

A suíte de testes está dividida em dois projetos principais:

Projeto de teste	Quantidade	Cobertura
Banco/.../Tests/	69	Repositórios, jobs, controllers
Hardware/.../BiometricAccess.Worker.Tests/	19	EventProcessor, conector Anviz, sincronizador T50

O framework é **xUnit + xUnit.runner.visualstudio**, com Microsoft.EntityFrameworkCore.Sqlite configurado para banco em memória (Data Source=:memory:) garantindo isolamento entre testes — cada teste cria seu próprio banco efêmero, popula dados específicos e desconta no fim.

Os testes do Int1 cobrem fluxos críticos: cadastro de pessoa com cifragem AES da senha (verifica que senhaClear não fica em plaintext), validação de CPF único, busca de senha disponível (verifica que triviais não são retornadas), execução do InativarUsuariosInativos2AnosJob (cria pessoa com dataUltimoAcesso antigo e confirma que vira inativo após executar), execução do LimparDadosExpiradosJob, e endpoints dos controllers principais.

Os testes do Worker cobrem o roteamento de eventos do EventProcessor: dado um EventoAcesso recebido do T50M, qual TentativaAcesso é gerada? Há casos cobrindo pessoa não cadastrada, pessoa inativa, pessoa sem permissão, biometria inválida, senha inválida, e acesso bem-sucedido por digital e por senha. Há também testes do SincronizadorT50Worker com mock do IAnvizService — verifica que a fila T50Pendencia é processada corretamente e que pendências com 5 falhas saem do round-robin.

Rodar todos os testes:

```
dotnet test Banco/WebAbil8-Sistema_Verificação_dupla.slnx/
dotnet test "Hardware & Serviço de Background/BiometricAccess.Worker.Tests/"
```

14. Build, Empacotamento e Execução

Pré-requisitos

- .NET 8 SDK
- Windows 10/11, Linux ou macOS no servidor (T50M é acessado via TCP/IP, então OS-agnóstico)
- Acesso à rede local onde os T50Ms estão conectados

Build

A partir da raiz do projeto:

```
dotnet build Banco/WebAbil8-Sistema_Verificação_dupla.sln/  
dotnet build PainelWeb/Frontend/  
dotnet build InfraestruturaBloco1/  
dotnet build "Hardware & Serviço de Background/BiometricAccess.Worker/BiometricAccess.  
Worker/"
```

Ou tudo de uma vez referenciando o .sln (se houver um). Como há dependências de projeto (Int3 → Int1, Int4 → Int1, Int2 → Int1 e Int4), buildar qualquer um traz as transitivas, mas buildar os quatro explicitamente é a forma mais segura de garantir que nenhuma cache do MSBuild esteja escondendo um erro.

Execução em desenvolvimento

O script `iniciar.ps1` na raiz do projeto orquestra a subida dos três serviços em sequência:

1. Mata processos antigos nas portas 5018 (Int1) e 8080 (Int3).
2. Faz backup do banco (cópia de `banco.db` para `banco_backup.db`).
3. Sobe Int1 escondido (sem janela), espera porta 5018 responder.
4. Sobe Int2 escondido.
5. Sobe Int3 escondido, espera porta 8080 responder.
6. Abre o navegador em `http://localhost:8080`.

Esse fluxo é interativo — o admin abre o PowerShell na pasta do projeto, executa `.\iniciar.ps1` e em poucos segundos está logado no painel. Para produção, a recomendação é instalar os três como serviços do Windows (NSSM) ou unidades do systemd no Linux. Detalhes completos em `docs/DeploymentGuide.html` seção 9.

Execução em produção

Os passos de produção, incluindo IP fixo do servidor, abertura de firewall, configuração SMTP, cadastro de T50M, instalação como serviço e backup automático, estão integralmente documentados em `docs/DeploymentGuide.html`.

Apêndice A — Glossário

Termo	Significado
Captive dependency	Singleton consumindo Scoped — anti-pattern resolvido com <code>IServiceScopeFactory</code>

Termo	Significado
EmployeeId	Identificador de usuário no T50M (= <code>CodigoUsuario</code> na nossa Pessoa)
OAE	Open Anviz Ethernet — protocolo proprietário Anviz
WAL mode	Write-Ahead Logging do SQLite — permite leitura concorrente durante escrita
Soft delete	Marca registro como <code>Excluido=true</code> sem remover fisicamente
Live edge	Ponto atual de um stream live (HLS)
Backbuffer	Memória do player com segmentos passados pra permitir seek
Replay RTSP	URL RTSP especial que retorna gravação histórica em vez de live
FAR/FRR	False Acceptance Rate / False Rejection Rate — métricas biométricas
JWT	JSON Web Token — token assinado para autenticação stateless
ONVIF	Open Network Video Interface Forum — padrão de interoperabilidade de câmeras IP
RTSP	Real Time Streaming Protocol — protocolo padrão de streaming de câmeras IP
HLS	HTTP Live Streaming — protocolo de streaming sobre HTTP, roda em navegadores
AES-CBC	Advanced Encryption Standard, modo Cipher Block Chaining

Apêndice B — Códigos de motivo de negação

Coluna `TentativaAcesso.MotivoNegacao` aceita:

Código	Quando ocorre
<code>nao_cadastrado</code>	ID/ <code>CodigoUsuario</code> não existe no banco
<code>inativo</code>	Pessoa existe mas <code>Status = "inativo"</code>
<code>sem_permissao</code>	Pessoa não tem <code>AmbientePessoa</code> neste ambiente
<code>biometria_invalida</code>	Match de digital falhou no T50M
<code>senha_invalida</code>	Senha digitada não confere com <code> Pessoa.senhaClear</code> decifrado
<code>biometria_nao_cadastrada_neste_t50m</code>	Pessoa tem <code>modoAcesso=digital_e_senha</code> mas não tem entrada em <code>PessoaT50</code> pra esse dispositivo (pendência de sincronização ainda não drenada)
<code>t50_lotado</code>	T50M atingiu 1000/1000 templates
<code>erro_interno</code>	Falha ao decifrar senha (AES) ou outra exceção